ORIGINAL CONTRIBUTION

Aadhaar Intel Engine: Turning Enrolment and Update Data into Something Operators Can Actually Use

1Author Name, 2Author Name
1Department of Computer Science and Engineering, Haldia Institute of Technology, Haldia, West Bengal
2UG Student, Dept. of CSE, Haldia Institute of Technology, Haldia, West Bengal

(Received Date: __ _____, 2026; Acceptance Date: __ _____, 2026)

ABSTRACT

Aadhaar generates huge day-to-day operational data—new enrolments, biometric updates, demographic updates—but messy place names and unevaluated "AI" charts make it hard to use. The Aadhaar Intel Engine cleans first, then analyses honestly: multi-million-row CSVs, durable geo repair with an audit trail, Parquet daily marts, and a Streamlit app (dashboard, analytics, forecast, light-basemap map, governance). Risk uses Isolation Forest on multi-feature state×district cells plus a state risk radar ranked by flagged volume, with volume printed next to each state name. Demand forecasting compares Seasonal, Linear Ridge, moving-average, and holiday-aware models under rolling-origin CV and keeps the best sMAPE only when it beats the moving-average baseline, with residual/conformal bands. A local LLM (Ollama) may write notes, but every number comes from the engine. On March–December 2025 marts (~63k / ~74k / ~80k enrol/bio/demo rows; ~5.4M enrolment volume), reloads are under a second after first build; Linear Ridge often won the bake-off. This is not a fraud detector; forecasts are guides. Figures are from a live run.

KEYWORDS: Aadhaar; data quality; anomaly detection; risk radar; forecasting; Isolation Forest; geospatial analytics; governance; LLM

1. INTRODUCTION

Anyone who has opened a government CSV knows the feeling: useful numbers are there, but the labels fight you. The same place shows up as Bangalore and Bengaluru. A district name lands in the state column. Telangana rows still say Andhra Pradesh. On top of that, many dashboards wave "AI" at the problem without saying how the model was checked, and dark maps make print-friendly reporting harder.

We wanted something closer to how a careful analyst would work. Clean the geography and keep an audit trail when humans fix names. Score unusual districts with more than one signal, then roll those flags up to a state risk radar ordered by volume so busy operators see where volume and oddity meet. Forecast enrolment with models that report error bands, not only a pretty curve. Map volume on a white basemap. And if a language model writes a summary, make sure it cannot invent the statistics.

That is the idea behind Aadhaar Intel Engine. The rest of the paper walks through how it works, what we saw on a full extract, and where we would not trust it yet. Screenshots are from the live Streamlit site; charts were exported from the same engine the site uses.

2. RELATED WORK

Public dashboards are good at totals and maps, less good at data repair. Forecasting textbooks tell us to compare models on held-out data rather than trust a single story [2], [3], [12]. Isolation Forest is a familiar tool for multi-feature outliers [1], [6]. Place-name quality is an old data-cleaning problem [5], [7], [11]. Local LLMs are convenient for write-ups [9], but they need guardrails. We put these pieces in one pipeline for Aadhaar-style aggregates [4], [8], [10].

3. HOW THE SYSTEM WORKS

3.1 What goes in

We read three kinds of aggregate files: enrolments (with age bands), biometric updates, and demographic updates. Each row has a date, state, district, and pincode. For anomalies we look at state × district. For forecasting we sum enrolments to a national daily series. After cleaning, our working marts cover March–December 2025 with about 63k enrolment, 74k biometric, and 80k demographic daily rows (~5.4M total enrolment volume; ~70M biometric and ~49M demographic update events).

3.2 Pipeline in plain terms

Figure 1 is the big picture. Files are classified by columns, cleaned, and collapsed when the same key appears twice. Then we save daily tables so the app does not re-read hundreds of megabytes every time. Inside the app you get a research dashboard, analytics (including the state risk radar), forecast, light-basemap geospatial view, and a governance screen for place-name fixes.

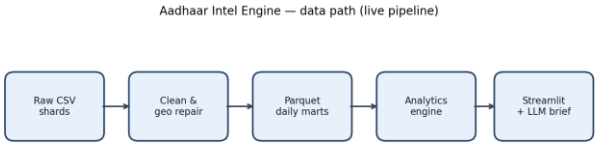

Fig. 1. Live pipeline sketch: raw CSVs → clean/geo repair → Parquet marts → analytics → Streamlit + LLM brief.

3.3 Fixing place names

We map state aliases (Orissa → Odisha, and so on). When someone typed a district into the state field (Jaipur, Nagpur, Darbhanga), we try to put it back under the right state. We also normalise common district spellings, use PIN-prefix hints where available, and move known Telangana districts off Andhra Pradesh when they were tagged wrongly. Human overrides are recorded so repairs stay durable across reloads rather than vanishing on the next CSV ingest.

3.4 Finding odd cells and the risk radar

Per state–district cell we use log volume, day-to-day volatility, bio/demo ratios, recent growth, active days, and volume versus the state median. Isolation Forest flags outliers and records which feature stood out. District flags then roll up to a state-level risk radar: the top ten

states by flagged volume, with max and mean risk on the radial axis and compact volume printed next to each state name on the chart. We still label the result as "look here," not "fraud."

3.5 Forecasting without the theatre

We try four simple models: day-of-week seasonal baseline, linear Ridge trend, a 7-day moving average, and a holiday-aware seasonal baseline. Models are scored with rolling-origin cross-validation (MAPE, sMAPE, RMSE). Auto mode keeps the lowest sMAPE only when it beats the moving-average baseline; otherwise the safer average is retained. The chart shows a point path plus residual/split-conformal uncertainty bands and a decision band label (tight / directional / exploratory).

3.6 Maps operators can print

The geospatial module uses a light Carto Positron basemap with dark state borders and volume-scaled markers (2D intensity, heatmap, or 3D columns). District centroids are preferred when available; otherwise state centroids with mild jitter. Tooltips and export stay aligned with the same cleaned marts.

3.7 Writing that cannot invent numbers

Research briefs follow Finding / Interpretation / Evidence / Method / Limitations. Evidence always comes from code. If Ollama is running, the model only writes the prose around those facts.

4. WHAT WE SAW

4.1 After cleaning

On the full extract, non-official "states" were cleared from the marts. Daily tables shrank to tens of thousands of rows, which is why reloads feel instant after the first build (Table 1).

Item	Rough size / time
Raw rows (all three datasets, pre-mart)	~4.9 million
Date span	Mar–Dec 2025
Enrol / bio / demo daily rows after clean	~63k / ~74k / ~80k
Enrolment volume in mart	~5.4 million
Active states (official set)	36
First build of marts	about 15–45 s
Later load from cache	under 1 s

Table 1. Scale on our workstation runs (rounded; live marts).

4.2 Live app and workload

Figure 2 is a screenshot from the running Research Dashboard. KPIs show total enrolments across age bands, biometric and demographic update totals, risk cell count, and a short forecast delta with holdout/rolling error. Figure 3 is the live workload mix from the same engine (not a stock image): enrolment volume is a small fraction of total update traffic, which is expected for a mature ID system.

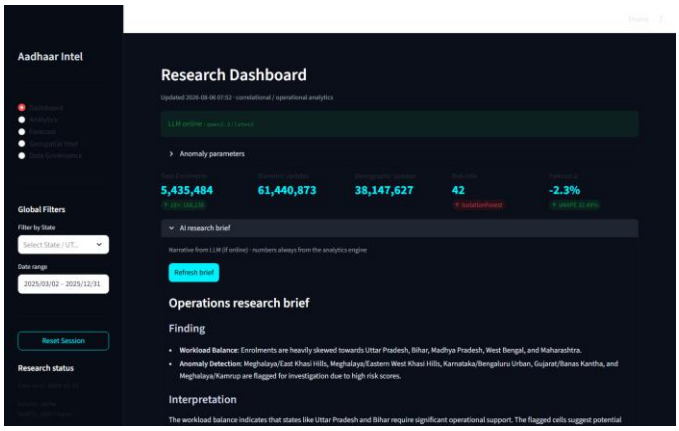

Fig. 2. Live Streamlit Research Dashboard (screenshot from the running app).

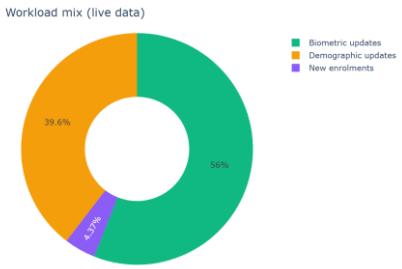

Fig. 3. Live workload mix chart (same engine as the website).

4.3 Risk radar, district flags, and forecast

Figure 4 is the live state risk radar: top ten states by flagged volume, with volume on the angular labels and risk score on the radial axis. Figure 5 ranks individual high-risk state×district cells from the same Isolation Forest pass (about 44 flags under contamination 0.05 and min volume 50 in our run). Table 2 and Figure 6 show the rolling bake-off and national daily forecast; Linear Ridge led with rolling sMAPE ≈ 41.6% versus ≈ 50.5% for the 7-day moving average and higher for seasonal baselines. Figure 7 is the Forecast module UI; Figure 8 is Geospatial Intel on the white basemap (live app at http://127.0.0.1:8501; not stock images).

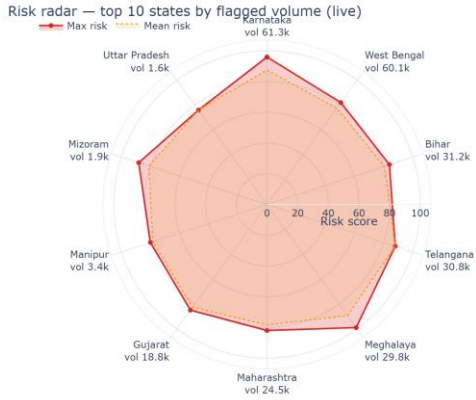

Fig. 4. Live risk radar — top 10 states by flagged volume (max/mean risk; volume on labels).

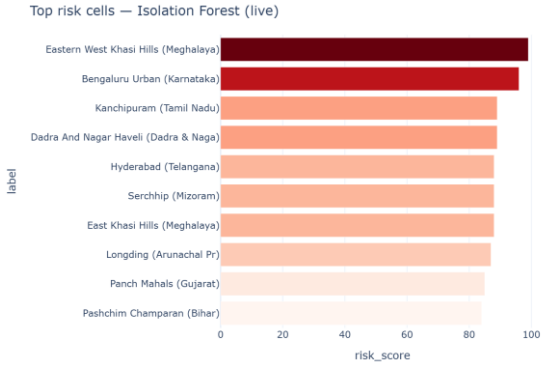

Fig. 5. Live top risk cells (Isolation Forest on composite state×district features).

Model	sMAPE %	MAPE %	RMSE (approx.)	Folds
Linear Ridge	41.6	212.1	34.5k	4
Moving average (7-day)	50.5	172.1	37.4k	4
Seasonal + holiday	58.1	340.0	40.4k	4
Seasonal (DOW × level)	64.9	352.7	48.3k	4

Table 2. Rolling-origin holdout comparison on national daily total enrolments (live session, rounded). Lower sMAPE is better. Auto mode selects Linear only when it beats MA.

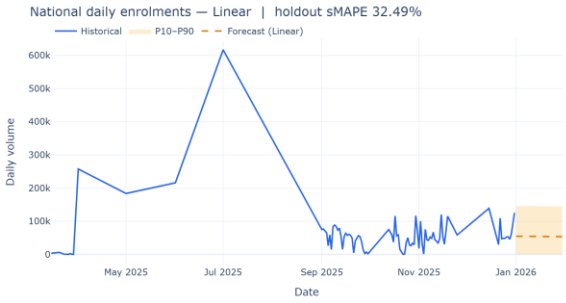

Fig. 6. Live forecast chart (history + selected model + uncertainty band).

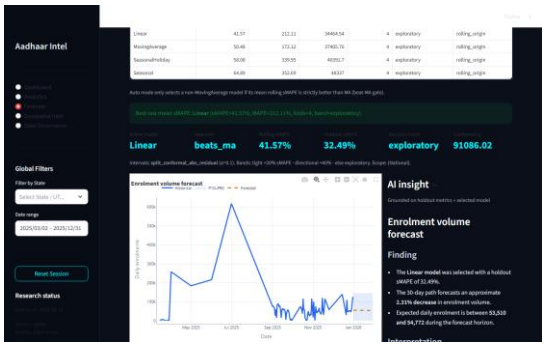

Fig. 7. Live Forecast module screenshot (app UI).

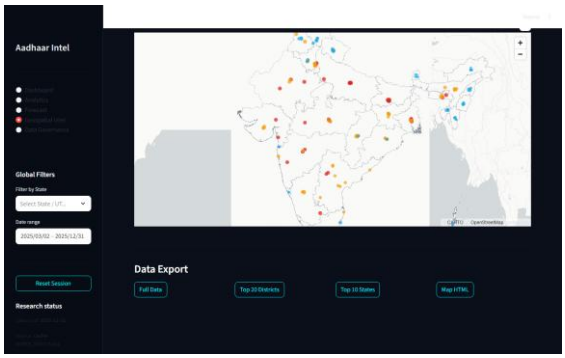

Fig. 8. Live Geospatial Intel screenshot (light/white basemap).

5. LIMITATIONS (THE HONEST PART)

Name repair is better than nothing, but it is not a full government gazetteer. Forecast error is still large on some windows—use the band and the decision label, not only the middle line. Anomaly and risk radar scores are unsupervised volume-weighted rankings, not fraud labels. We have no population or centre-capacity denominator. LLM text can still sound too sure of itself; when in doubt, read the Evidence block. Screenshots reflect one live session and will change with filters and data refreshes.

6. CONCLUSION

Aadhaar Intel Engine tries to treat operational identity data with a bit more care: clean first, measure models, explain flags, put volume next to risk on a state radar, map on a light basemap, and keep language models on a short leash. The figures in this paper come from that live path, not from stock illustration folders. Next steps we care about most are official district masters, rates per population or centre, tighter forecast intervals, and multi-user access control.

Acknowledgement. Built for education and symposium discussion. Aggregates only—no Aadhaar numbers or biometrics. Replace author names before submission. Live assets regenerated with docs/capture_live_assets.py.

REFERENCES

[1] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. ICDM, 413–422.
[2] Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.
[3] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and ML forecasting methods. PLOS ONE, 13(3).
[4] UIDAI. Aadhaar documentation. https://uidai.gov.in/
[5] Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. IEEE Data Eng. Bull., 23(4).
[6] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Comput. Surv., 41(3).
[7] MoPR. Local Government Directory. https://lgdirectory.gov.in/
[8] Streamlit docs. https://docs.streamlit.io/
[9] Ollama. https://ollama.com/
[10] Pedregosa, F., et al. (2011). Scikit-learn. JMLR, 12, 2825–2830.
[11] Batini, C., & Scannapieco, M. (2016). Data and Information Quality. Springer.
[12] Box, G. E. P., et al. (2015). Time Series Analysis: Forecasting and Control (5th ed.). Wiley.